Lab 4

Lab 4 Report

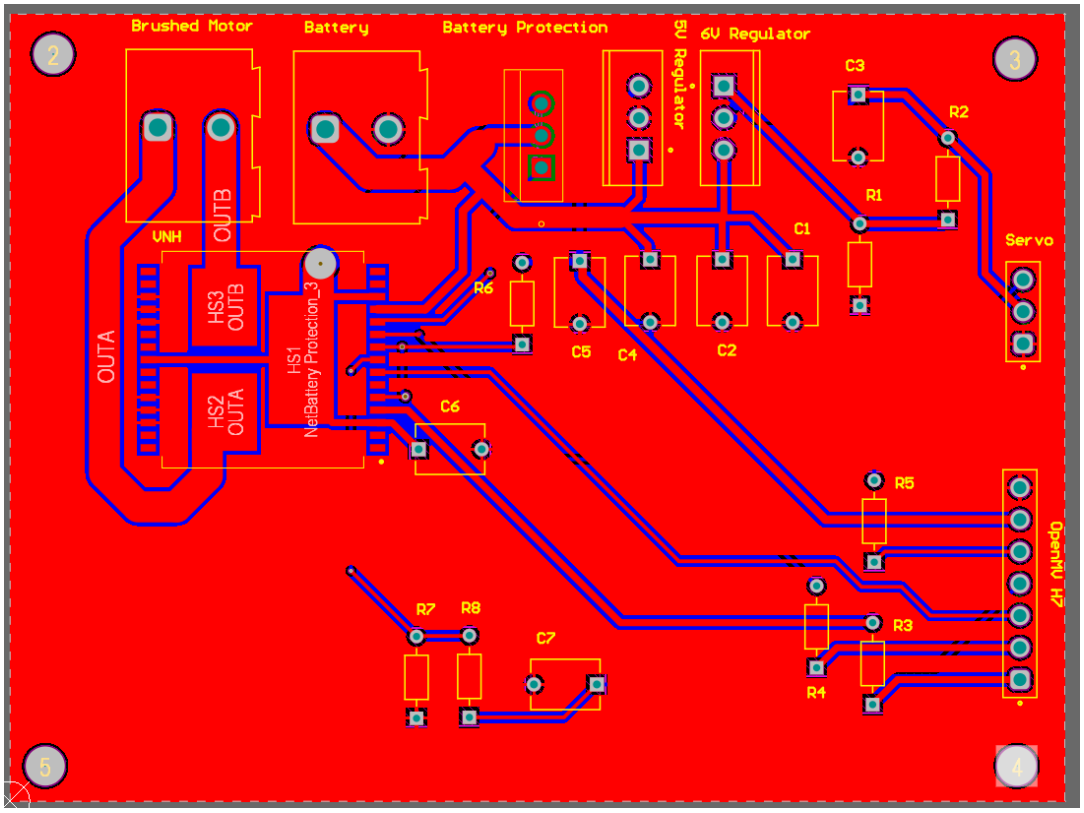
Nicolas Wong-Shea, Temirlan Ryspaev, Xiang Mao, Justin Spenker

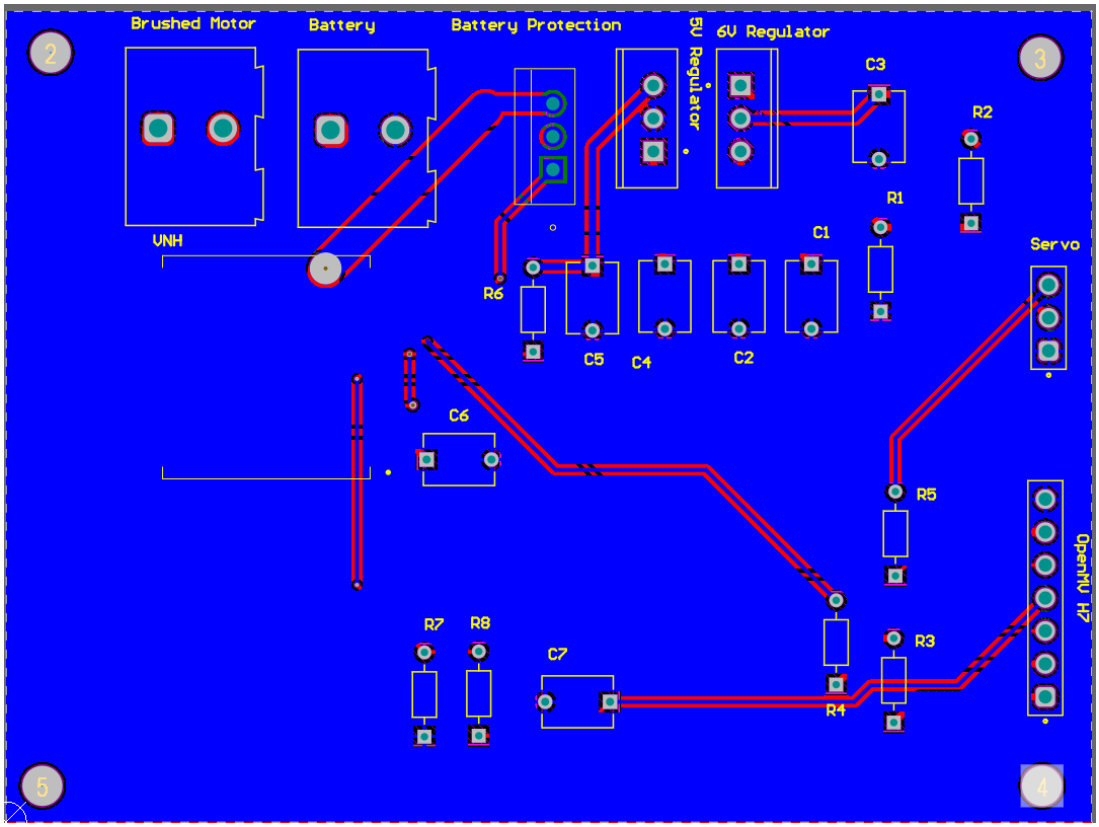
Each group member was responsible for working on a part of the PCB. Every group member contributed in a discussion style on placement, wiring, and parameter adjustments. We came together as a group to troubleshoot and design to create the final design. Everyone contributed roughly equally to the project.

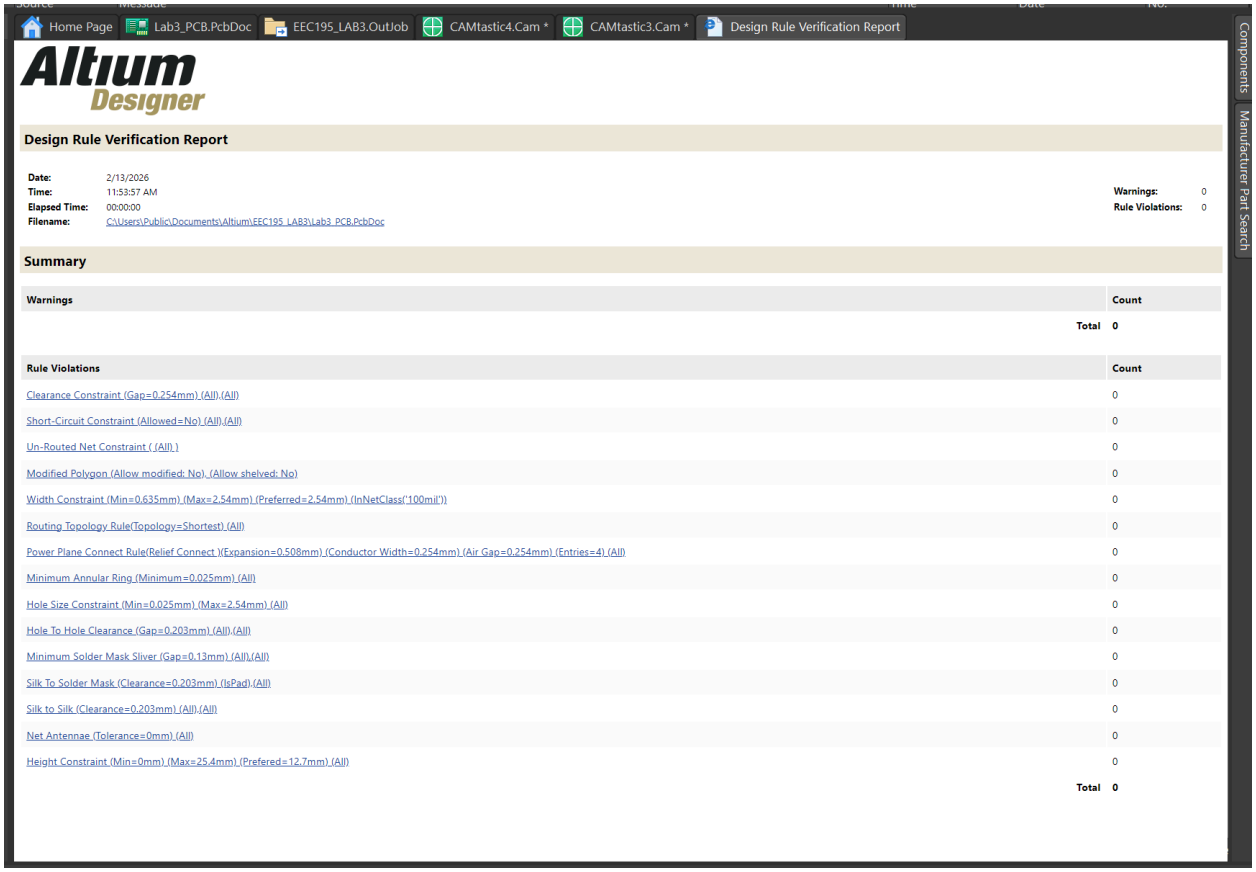
EEC195A Lab #: 4

Names: Justin Spenker
Nicolas Wong-Shea
Temirlan Ryspaev
Xiang Mao

	Score	TA signature / date
PCB	✓	Feb 13
TOTAL LAB SCORE		


Brushed Motor
Battery
Battery Protection
5V Regulator
6V Regulator
C3
R2
R1
C1
Servo
VNH
OUTB
HS3
OUTB
OUTA
HS1
NetBattery Protection_3
HS2
OUTA
R6
C5
C4
C2
C6
R5
OpenMV H7
R7
R8
C7
R3
R4
2
3
5
4


Brushed Motor
Battery
Battery Protection
5V Regulator
6V Regulator
C3
R2
R1
C1
Servo
VNH
R6
C5
C4
C2
C6
R5
OpenMV H7
R7
R8
C7
R3
R4
2
3
5
4



DFM Checks

The minimum distance values from the design data are displayed and referenced against our Standard & Advanced capabilities. Whenever possible, designing according to our Standard capabilities will limit manufacturing costs, lead time, and quality concerns.

	Min. Distance	Bay Area Circuits Standard Capability	Bay Area Circuits Advanced Capability
copper to board edge:	0.0 mil (0 mm)	10 mil	0 mil
copper to plated hole:	12.5 mil (0.318 mm)	10.5 mil	5 mil
copper to non-plated hole:	-	8 mil	4 mil
copper to copper:	10.0 mil (0.254 mm)	6 mil	2 mil
copper ring:	2.5 mil (0.064 mm)	6 mil	1 mil
track width:	15.0 mil (0.381 mm)	6 mil	2 mil

Green = Standard Capability, Orange = Advanced Capability, Red = Not Within Capability

Potential Show Stoppers

Insufficient Annular Ring (10 violations) 1 2 3 4 5

Problems Automatically Fixed

Insufficient Soldermask Clearance (66 violations) 1 2 3 4 5

NOTE: The insufficient annular ring violation can be easily modified to support AdvancedPCB production. I had catered the design for the BayArea PCB. Not a problem for final design.

Lab 5

Lab 5 Report

Nicolas Wong-Shea, Temirlan Ryspaev, Xiang Mao, Justin Spenker

Team Participation Report

For this lab everyone in the group participated in group discussion and troubleshooting. Everyone in the group did an equal amount of work and helped each other complete this lab.

Summary

We first tested the rustler car to confirm its functionality. Next, we successfully soldered the PCB components and interface pins onto the circuit board. During soldering, our team encountered an issue: overheating during pin soldering caused pin displacement and bulging. When we removed the pins for adjustment, solder flowed into the holes and clogged them. We resolved this by reheating the solder joints and reinserting the pins after the solder melted.After soldering, we used a multimeter to check the PCB connections, confirming all connections were correct. In the third part, we updated the OpenMV firmware. The only issue was that the micro-USB cable we initially used only supported power; we replaced it with a cable that supported data transfer to complete the update.

In Part 4, we first designed the servo and esc interfaces as P7, 8, to connect to CH1, 2 on the OpenMV, then adjusted the timing and wrote our test program. However, during testing, data transmission and connections were unstable, requiring manual reinforcement. We initially suspected a cable connection fault. Upon inspection, our technicians found that some OpenMV pins were not fully soldered, so we had to manually adjust them during testing. The problem was resolved after resoldering the pins. In Part 5, we noticed that the code couldn't control the vehicle to reverse twice consecutively. Initially, we thought this was a safety mechanism. However, they explained that this safety mechanism was due to the vehicle not supporting true braking in remote control mode; braking was achieved through a brief reverse pulse. Therefore, to reverse again, we needed to insert an additional "brake + reverse" sequence to reset the ESC state. After modifying the code, we successfully implemented the expected reverse function, and the experiment proceeded smoothly without any other major problems

EEC195A Lab #: 5

Names: Justin Spenker

Nicolas Wong-Shea

Temirlan Ryspaev

Xiang Mao

	Score	TA signature / date
Part 4 Part 5. test 1 Part 5. test 2	✓ ✓ ✓	Feb20 Feb20 Feb20
TOTAL LAB SCORE		

Python code

```python
import time
from pyb import Pin, Timer

tim = Timer(4, freq=100)
servo = tim.channel(1, Timer.PWM, pin=Pin("P7"))
DC    = tim.channel(2, Timer.PWM, pin=Pin("P8"))

while True:
    print("straight + brake (10s)")
    servo.pulse_width(2880)
    DC.pulse_width(2880)
    time.sleep_ms(10000)

    print("right + forward (2s)")
    servo.pulse_width(2112)
    DC.pulse_width(3168)
    time.sleep_ms(2000)

    print("straight + forward (2s)")
    servo.pulse_width(2880)
    DC.pulse_width(3168)
    time.sleep_ms(2000)

    print("left + forward (2s)")
    servo.pulse_width(3648)
    DC.pulse_width(3168)
    time.sleep_ms(2000)

    print("straight + brake (1s)")
    servo.pulse_width(2880)
    DC.pulse_width(2880)
    time.sleep_ms(1000)

    print("straight + reverse (0.5s)")
    servo.pulse_width(2880)
    DC.pulse_width(2400)
    time.sleep_ms(500)

    print("straight + brake (0.5s)")
    servo.pulse_width(2880)
    DC.pulse_width(2880)
    time.sleep_ms(500)

    print("straight + reverse (1s)")
    servo.pulse_width(2880)
    DC.pulse_width(2400)
    time.sleep_ms(1000)

    print("straight + brake (1s)")
    servo.pulse_width(2880)
    DC.pulse_width(2880)
    time.sleep_ms(1000)

    print("straight + reverse (2s)")
    servo.pulse_width(2880)
    DC.pulse_width(2496)
    time.sleep_ms(2000)

    print("straight + brake (1s)")
    servo.pulse_width(2880)
    DC.pulse_width(2880)
    time.sleep_ms(1000)
```

Lab6

Lab 6 Report

Nicolas Wong-Shea, Temirlan Ryspaev, Xiang Mao, Justin Spenker

Team Participation Report

For this lab everyone in the group participated in group discussion and troubleshooting. Everyone in the group did an equal amount of work and helped each other complete this lab.

Summary

The first part is relatively simple. We measured the default frame rate and then improved performance by switching to grayscale and QQVGA modes. We toggled the red LED every 50 iterations in the main loop, which also helped confirm that the camera loop was running at a stable rate. The second part required the most debugging. In Program 1, our initial blob detection was unstable: it couldn't detect multiple blobs at once. Threshold parameters and pixel filters caused the algorithm to either lock onto a single specular artifact or completely miss ground lines. Program 3 initially calculated the angle between the center and one of the blobs, rather than the angle between the two blobs. We corrected the geometry by calculating the deflection angle based on the relative positions of the two detected blobs, thus achieving the expected motion trajectory of -45° to +45°.

Parts 3 and 4 are directly based on the same detection process. Part 3 integrates the visual output into the PWM control logic for steering and motor commands, and verifies it using LED status indicators and serial printing. Part 4 uses a regression-based deflection angle calculation. After parameter adjustments, both components functioned as expected. The remaining primary sensitivity lay in the white threshold's dependence on ambient light; we temporarily mitigated this issue by lowering the threshold and using controlled illumination during testing to improve contrast.

EEC195A Lab #: 6

Names: Justin Spenker

Nicolas Wong-Shea

Temirlan Ryspaev

Xiang Mao

	Score	TA signature / date
Part 2 P1 P2 P3 Part 3 Part 4	✓ ✓ ✓ ✓ ✓	Feb 27
TOTAL LAB SCORE		

Python code

```
# Team members: Justin Spenker, Nicolas Wong-Shea
#               Temirlan Ryspaev, Xiang Mao
# Lab 6 Part 1 Code: Print average fps and toggle LED every 50 cc

import sensor
import time
from pyb import LED

sensor.reset()  # Reset and initialize the sensor.
sensor.set_pixformat(sensor.GRAYSCALE)  # Set pixel format to RGB565 (or GRAYSCALE)
sensor.set_framesize(sensor.QQVGA)  # Set frame size to QVGA (320x240)
sensor.skip_frames(time=2000)  # Wait for settings take effect.
clock = time.clock()  # Create a clock object to track the FPS.

rled = LED(1)
count = 0

while True:
    clock.tick()  # Update the FPS clock.
    img = sensor.snapshot()  # Take a picture and return the image.
    print(clock.fps())  # Note: OpenMV Cam runs about half as fast when connected
    # to the IDE. The FPS should increase once disconnected.
    count += 1
    if (count >= 50):
        rled.toggle()
        count = 0
```

Part 1 Code

```python
# Team members: Justin Spenker, Nicolas Wong-Shea
#               Temirlan Ryspaev, Xiang Mao
# Lab 6 Part 2  Program 1 Code: Find white blobs
#               using single ROI at top

import sensor, image, time, pyb

# LEDs
led_r = pyb.LED(1)

# Camera setup
sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QQVGA)   # 160x120
sensor.skip_frames(time=2000)
sensor.set_auto_gain(False)
sensor.set_auto_whitebal(False)

clock = time.clock()

# ROI
ROI_H = 10
W = 160
ROI = (0, 0, W, ROI_H)

# Blob thresholds
WHITE_THRESH = [(200, 255)]
MIN_PIXELS = 3
MIN_AREA   = 3

loop_count = 0

while True:
    clock.tick()
    img = sensor.snapshot()

    blobs = img.find_blobs(
        WHITE_THRESH,
        roi=ROI,
        pixels_threshold=MIN_PIXELS,
        area_threshold=MIN_AREA,
        merge=True
    ) or []

    # Draw ALL blobs (box + cross)
    for b in blobs:
        img.draw_rectangle(b.rect(), color=0)
        img.draw_cross(b.cx(), b.cy(), color=0, size=5)

    # Blink red LED every 50 frames
    loop_count += 1
    if loop_count >= 50:
        led_r.toggle()
        loop_count = 0

    # Print FPS
    print("FPS: %.1f  ROI_H: %d  Blobs: %d" % (clock.fps(), ROI_H, len(blobs)))
```

Part 2 Program 1

```python
# Team members: Justin Spenker, Nicolas Wong-Shea
#               Temirlan Ryspaev, Xiang Mao
# Lab 6 Part 2  Program 2 Code: Find blobs but place box
#               around only the centermost blob
#               and toggle LED every 50 cc
import sensor, image, time, pyb

# LED Setup
red_led = pyb.LED(1)

# Camera Setup
sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QQVGA)   # 160x120
sensor.skip_frames(time=2000)
sensor.set_auto_gain(False)
sensor.set_auto_whitebal(False)

clock = time.clock()

# ROI
# Experiment with height: start at 10, try 5 or 8 for higher FPS
ROI_HEIGHT = 10
ROI = (0, 0, 160, ROI_HEIGHT)

ROI_CENTER_X = 160 // 2   # x-coordinate of the ROI's center (80)

# Thresholds — reduce MIN_PIXELS/MIN_AREA if you shrink ROI height
WHITE_THRESHOLD = [(200, 255)]
MIN_PIXELS = 3
MIN_AREA   = 3

loop_count = 0

while True:
    clock.tick()
    img = sensor.snapshot()

    blobs = img.find_blobs(
        WHITE_THRESHOLD,
        roi=ROI,
        pixels_threshold=MIN_PIXELS,
        area_threshold=MIN_AREA,
        merge=True
    )

    if blobs:
        # Use lambda to find the blob whose cx is closest to ROI center
        center_blob = max(blobs, key=lambda b: -abs(b.cx() - ROI_CENTER_X))

        # Draw rectangle and cross ONLY on the center blob
        img.draw_rectangle(center_blob.rect(), color=0)
        img.draw_cross(center_blob.cx(), center_blob.cy(), color=0, size=5)

    # Toggle red LED every 50 frames
    loop_count += 1
    if loop_count >= 50:
        red_led.toggle()
        loop_count = 0

    fps = clock.fps()
    print("FPS: %.1f  ROI height: %d  Blobs: %d" % (fps, ROI_HEIGHT, len(blobs)))
```

Part 2 Program 2

```python
import sensor, image, time, pyb, math

led_r = pyb.LED(1)

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QQVGA)
sensor.skip_frames(time=2000)
sensor.set_auto_gain(False)
sensor.set_auto_whitebal(False)

clock = time.clock()

W = 160
IMG_CX = W // 2

ROI_H = 8
ROI_TOP    = (0, 5,   W, ROI_H)
ROI_BOTTOM = (0, 107, W, ROI_H)

WHITE_THRESH = [(150, 255)]
MIN_PIXELS = 3
MIN_AREA   = 3

def find_center_blob(img, roi):
    blobs = img.find_blobs(
        WHITE_THRESH,
        roi=roi,
        pixels_threshold=MIN_PIXELS,
        area_threshold=MIN_AREA,
        merge=True
    ) or []
    if not blobs:
        return None
    return min(blobs, key=lambda b: abs(b.cx() - IMG_CX))

loop_count = 0

while True:
    clock.tick()
    img = sensor.snapshot()

    top_b = find_center_blob(img, ROI_TOP)
    bot_b = find_center_blob(img, ROI_BOTTOM)

    if top_b and bot_b:
        img.draw_rectangle(top_b.rect(), color=0)
        img.draw_cross(top_b.cx(), top_b.cy(), color=0, size=5)

        img.draw_rectangle(bot_b.rect(), color=0)
        img.draw_cross(bot_b.cx(), bot_b.cy(), color=0, size=5)

        if bot_b.cy() < top_b.cy():
            top_b, bot_b = bot_b, top_b

        dx = top_b.cx() - bot_b.cx()
        dy = bot_b.cy() - top_b.cy()   # positive

        angle_deg = math.degrees(math.atan2(dx, dy))
        angle_deg = max(-45.0, min(45.0, angle_deg))

        print("FPS: %.1f  Deflection: %.1f deg" % (clock.fps(), angle_deg))
    else:
        print("FPS: %.1f  One or both blobs not found" % clock.fps())

    loop_count += 1
    if loop_count >= 50:
        led_r.toggle()
        loop_count = 0
```

Part 2 Program 3

```python
import sensor, image, time
from pyb import LED, Pin, Timer
sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QQVGA)
sensor.skip_frames(time=2000)
sensor.set_auto_gain(False)
sensor.set_auto_whitebal(False)
W = sensor.width()
H = sensor.height()

ROI = (0, int(H * 0.55), W, int(H * 0.45))
PWM_FREQ = 100
SERVO_PIN = Pin("P7")
MOTOR_PIN = Pin("P8")
t = Timer(4, freq=PWM_FREQ)
ch_servo = t.channel(1, Timer.PWM, pin=SERVO_PIN)
ch_motor = t.channel(2, Timer.PWM, pin=MOTOR_PIN)

PERIOD_US = int(1_000_000 / PWM_FREQ)

def us_to_duty_percent_int(pulse_us):
    duty = int(round(100 * pulse_us / PERIOD_US))
    if duty < 0: duty = 0
    if duty > 100: duty = 100
    return duty

def set_pwm_us(ch, pulse_us):
    ch.pulse_width_percent(us_to_duty_percent_int(pulse_us))
# LEDs
RED = LED(1); GRN = LED(2); BLU = LED(3)
def led_off():
    RED.off(); GRN.off(); BLU.off()
def led_state(state):
    led_off()
    if state == "LEFT":
        BLU.on()
    elif state == "CENTER":
        GRN.on()
    elif state == "RIGHT":
        RED.on()
    else:
        RED.on(); BLU.on()

def classify_x(x):
    if x < W/3:
        return "LEFT"
    elif x < 2*W/3:
        return "CENTER"
    else:
        return "RIGHT"

WHITE_THRESHOLD = (150, 255)
MIN_WHITE_PIXELS = 80
clock = time.clock()
# Safe init
set_pwm_us(ch_servo, SERVO_CENTER_US)
set_pwm_us(ch_motor, MOTOR_STOP_US)
led_state("LOST")
while True:
    clock.tick()
    img = sensor.snapshot()
    bin_img = img.copy()
    bin_img.binary([WHITE_THRESHOLD])
    bin_img.erode(1)
    bin_img.dilate(1)

    blobs = bin_img.find_blobs([(255, 255)], roi=ROI,
                               pixels_threshold=MIN_WHITE_PIXELS,
                               area_threshold=MIN_WHITE_PIXELS,
                               merge=True)
    if not blobs:
        state = "LOST"
        servo_us = SERVO_CENTER_US
        motor_us = MOTOR_STOP_US
        led_state(state)

        img.draw_string(2, 2, "LOST", color=255, scale=2)
    else:
        b = max(blobs, key=lambda bb: bb.pixels())

        img.draw_rectangle(b.rect(), color=255)
        img.draw_cross(b.cx(), b.cy(), color=255)
```

```python
    reg = bin_img.get_regression([(255, 255)], roi=ROI, robust=True)

    if reg is not None:
        img.draw_line(reg.line(), color=255)

        x1, y1, x2, y2 = reg.x1(), reg.y1(), reg.x2(), reg.y2()
        y_bottom = ROI[1] + ROI[3] - 1
        dy = (y2 - y1)
        if dy == 0:
            x_hit = (x1 + x2) / 2.0
        else:
            tpar = (y_bottom - y1) / dy
            x_hit = x1 + tpar * (x2 - x1)
    else:
        x_hit = b.cx()

    if x_hit < 0: x_hit = 0
    if x_hit > W - 1: x_hit = W - 1
    y_bottom = ROI[1] + ROI[3] - 1
    img.draw_cross(int(x_hit), int(y_bottom), color=255)

    state = classify_x(x_hit)
    img.draw_string(2, 2, state, color=255, scale=2)

    if state == "LEFT":
        servo_us = SERVO_LEFT_US
        motor_us = MOTOR_FWD_US
    elif state == "CENTER":
        servo_us = SERVO_CENTER_US
        motor_us = MOTOR_FWD_US
    else:
        servo_us = SERVO_RIGHT_US
        motor_us = MOTOR_FWD_US

    led_state(state)

# Apply PWM
set_pwm_us(ch_servo, servo_us)
set_pwm_us(ch_motor, motor_us)
servo_duty = us_to_duty_percent_int(servo_us)
motor_duty = us_to_duty_percent_int(motor_us)
print("state=%s servo=%dus (%d%%) motor=%dus (%d%%) FPS=%.1f"
      % (state, servo_us, servo_duty, motor_us, motor_duty, clock.fps()))
```

Part 3 Code

```python
# Team members: Justin Spenker, Nicolas Wong-Shea
#               Temirlan Ryspaev, Xiang Mao
# Lab 6 Part 4: Find lin regression line and compute
#               deflection angle relative to car.
#               Displays LED color depending on
#               which region the line is in.
import sensor, image, time, pyb, math

led_r = pyb.LED(1)
led_g = pyb.LED(2)
led_b = pyb.LED(3)

def leds_off():
    led_r.off(); led_g.off(); led_b.off()

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QQQVGA)
sensor.skip_frames(time=2000)
sensor.set_auto_gain(False)
sensor.set_auto_whitebal(False)

clock = time.clock()

W = sensor.width()
H = sensor.height()
third = W // 3

# Threshold for white line on dark background
WHITE_THRESH = [(140, 255)]

while True:
    clock.tick()
    img = sensor.snapshot()

    line = img.get_regression(WHITE_THRESH, robust=True)

    region_str = "None"
    angle_deg = None

    if line:
        x1 = line.x1(); y1 = line.y1()
        x2 = line.x2(); y2 = line.y2()

        # Draw the detected regression line
        img.draw_line(x1, y1, x2, y2, color=0)

        if y1 > y2:
            bx, by = x1, y1
            tx, ty = x2, y2
        else:
            bx, by = x2, y2
            tx, ty = x1, y1

        dx = tx - bx
        dy = by - ty  # positive

        angle_deg = math.degrees(math.atan2(dx, dy))

        # Endpoint region test (both endpoints must be in same third)
        xmin = x1 if x1 < x2 else x2
        xmax = x2 if x1 < x2 else x1

        leds_off()
        if xmax < third:
            region_str = "Left"
            led_b.on()
        elif xmin >= 2 * third:
            region_str = "Right"
            led_r.on()
        elif xmin >= third and xmax < 2 * third:
            region_str = "Center"
            led_g.on()
        else:
            region_str = "None"

        print("FPS: %.1f  Deflection: %.1f deg  Region:%s" % (clock.fps(), angle_deg, region_str))
    else:
        leds_off()
        print("FPS: %.1f  Deflection: None  Region:None" % clock.fps())
```

Part 4 Code

Lab7

Lab 7 Report

Nicolas Wong-Shea, Temirlan Ryspaev, Xiang Mao, Justin Spenker

Participation:
For this lab everyone in the group participated in group discussion and troubleshooting. Everyone contributed to the final schematic and pcb.

Lab Summary:
While creating the schematic, we opted to add options for bluetooth and speed sensing in case we wanted to implement them later. We also added another connector for additional OpenMV connections. This required adding 3 4-pin connectors to our design. We also opted to use the recommended high-current PCB connector, the B12 Lugs instead of the previous connectors. This caused some minor issues since the part had to be manually created. Overall, our design did not change that much from before.

For our PCB design, the layout had to change significantly due to the required use of a star ground, the additional connectors previously mentioned, and the replacement of the initial 2-pin terminal block connectors with the B12 lugs. Finally, they suggested placing capacitors close to the 5V and 6V regulators to improve input/output voltage stability and reduce noise and ripple.

Verification:

EEC195A Lab #: 7

Names: Justin Spenker

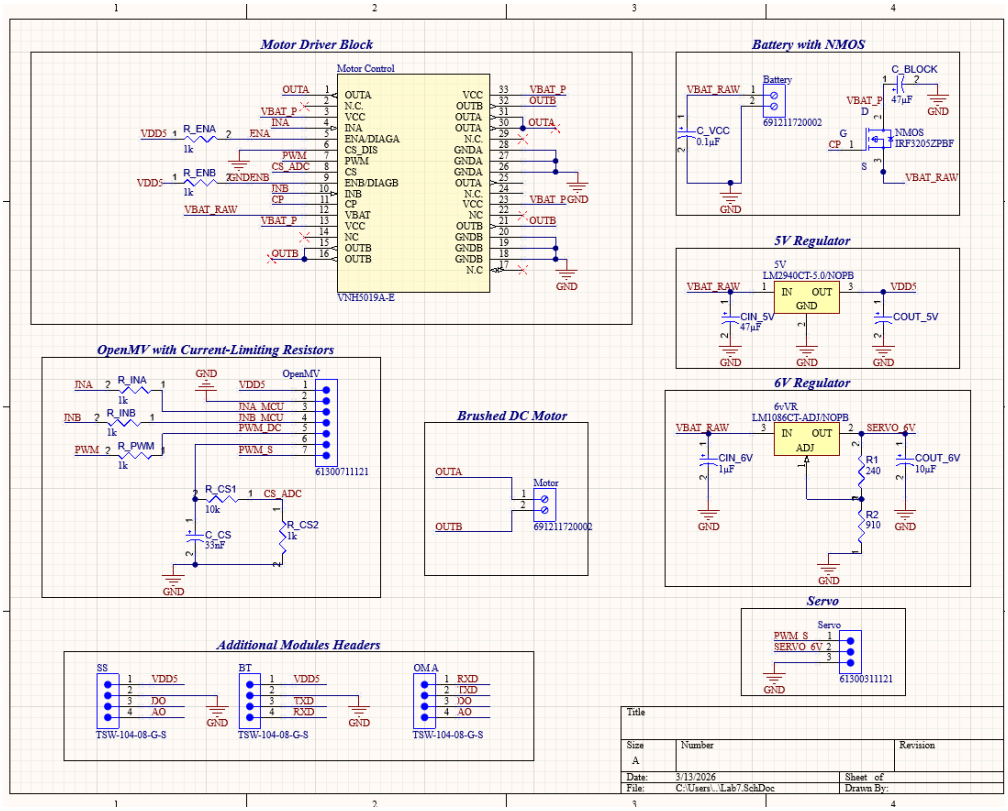
Nicolas Wong-Shea

Temirlan Ryspaev

Xiang Mao

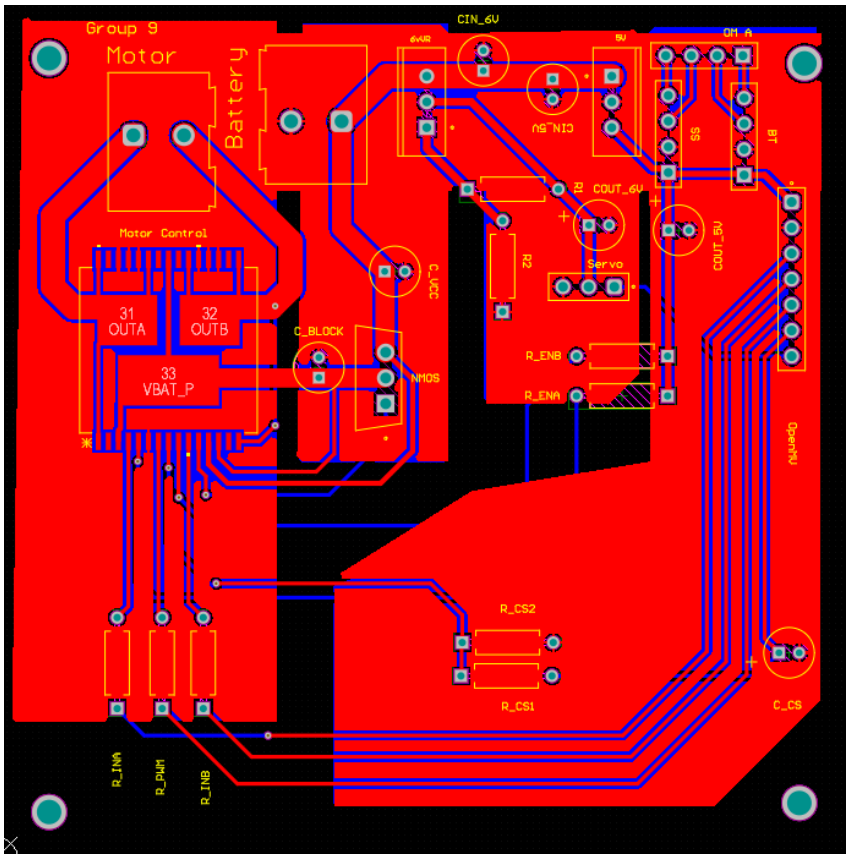
	Score	TA signature / date
Circuit Design	✓	Mar 6
PCB Design	✓	Mar 13
TOTAL LAB SCORE		

Schematic:

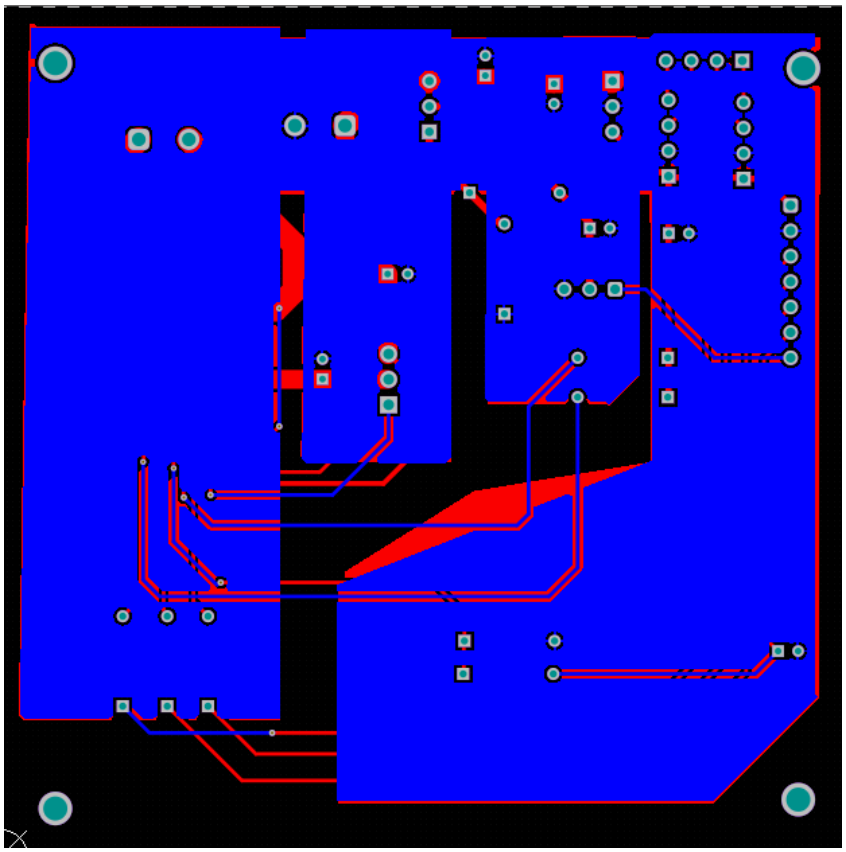


PCB:

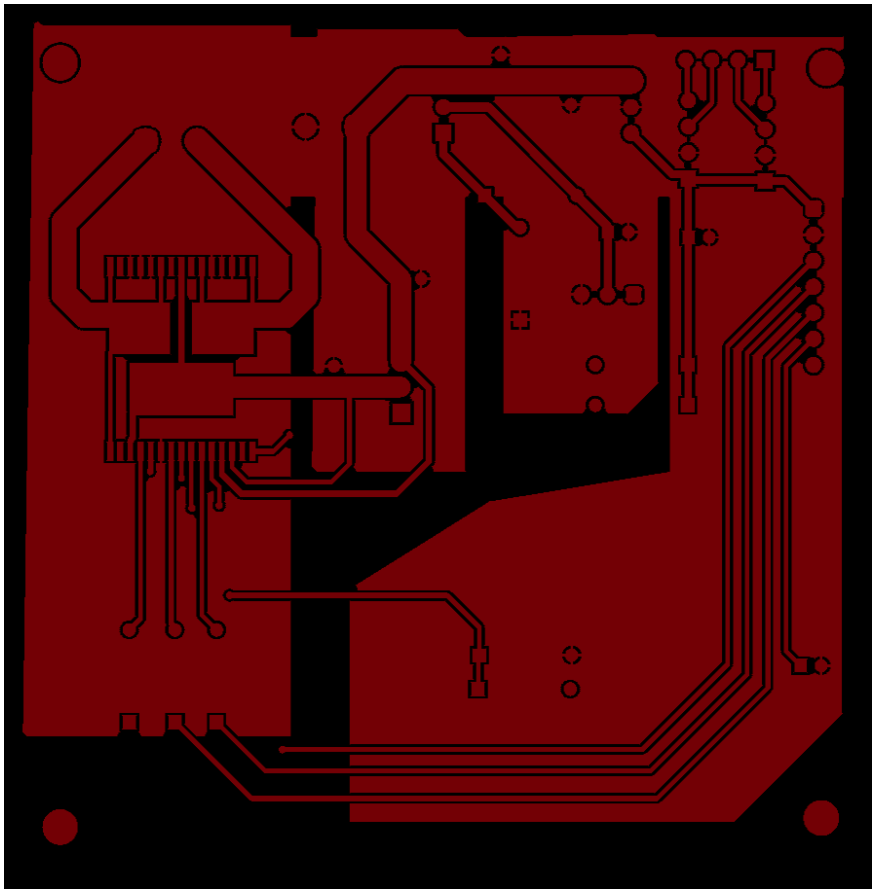
Top Layer:



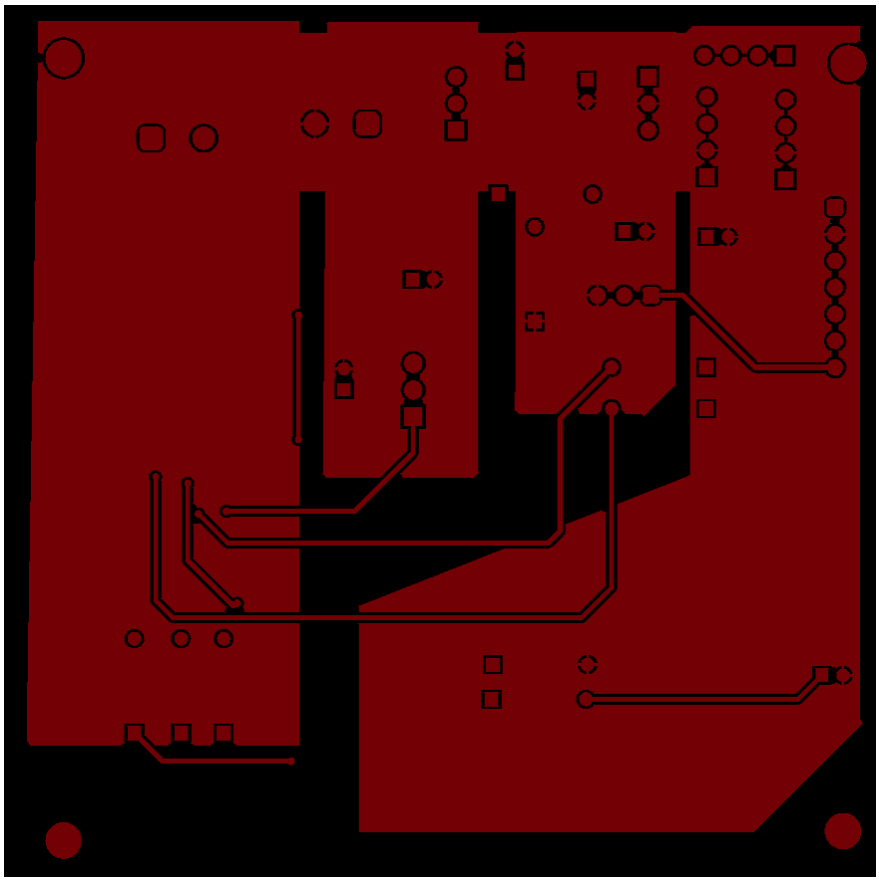
Bottom Layer:



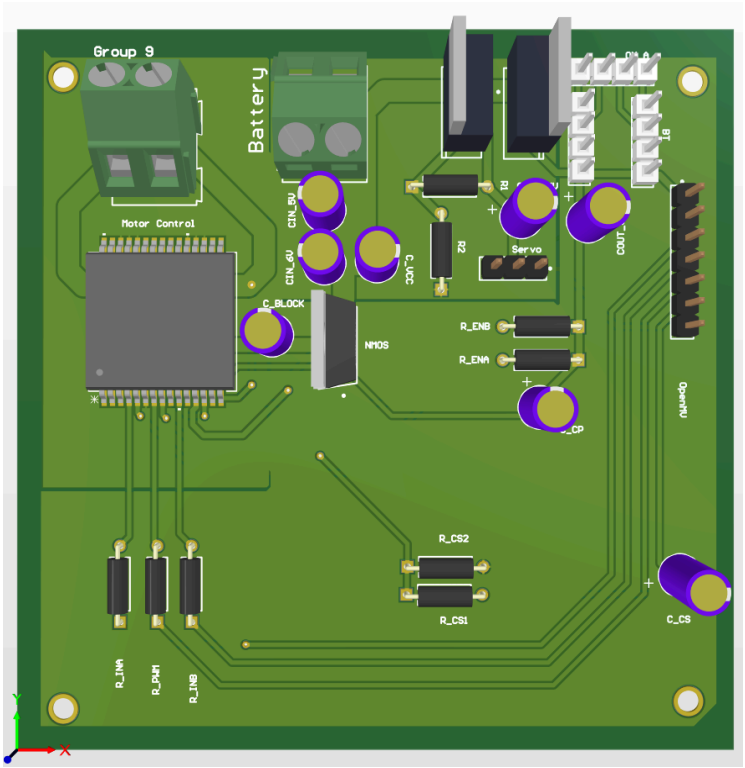
Top Copper Layer:



Bottom Copper Layer:



3D Version :


Group 9
Battery
Motor Control
CIN_5V
CIN_6V
C_VCC
C_BLOCK
NMOS
R1
R2
Servo
COUT
R_ENB
R_ENA
C_CP
OpenMV
R_CS2
R_CS1
C_CS
R_INA
R_PWM
R_INB

Validation:

[Info]	EEC1959ALab7.PrjF	Compiler	Compile successful, no errors found.	10:56:09 PM	3/13/2026	6

DRC:

Altium Designer

Design Rule Verification Report

Date:	3/13/2026		
Time:	10:19:32 PM	**Warnings:**	0
Elapsed Time:	00:00:00	**Rule Violations:**	0
Filename:	C:\Users\Public\Documents\Altium\EEC1959ALab4\Lab7.PcbDoc		

Summary

Warnings	Count
Total	**0**

BayArea DFM Check:

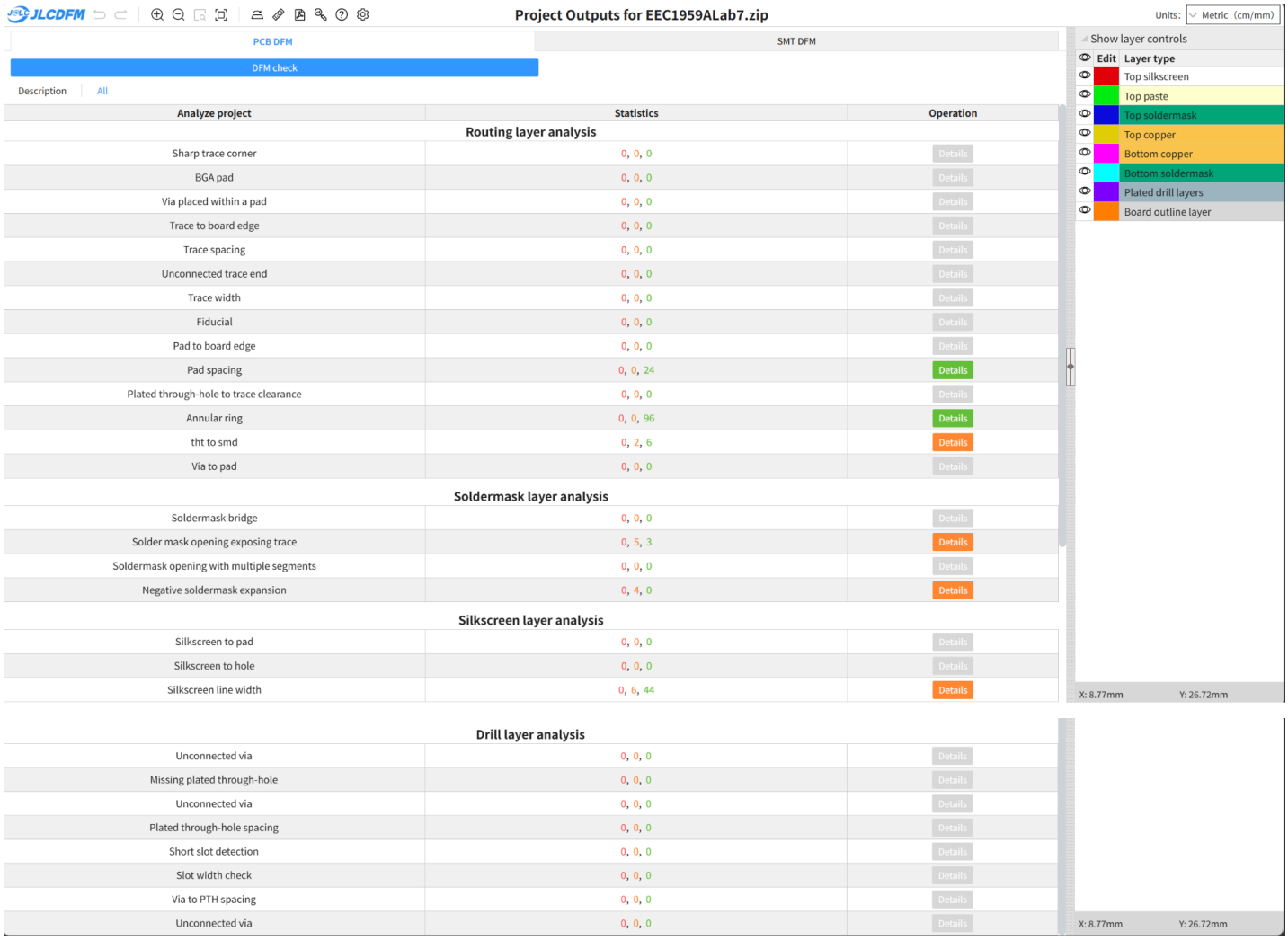
DFM Checks

The minimum distance values from the design data are displayed and referenced against our Standard & Advanced capabilities. Whenever possible, designing according to our Standard capabilities will limit manufacturing costs, lead time, and quality concerns.

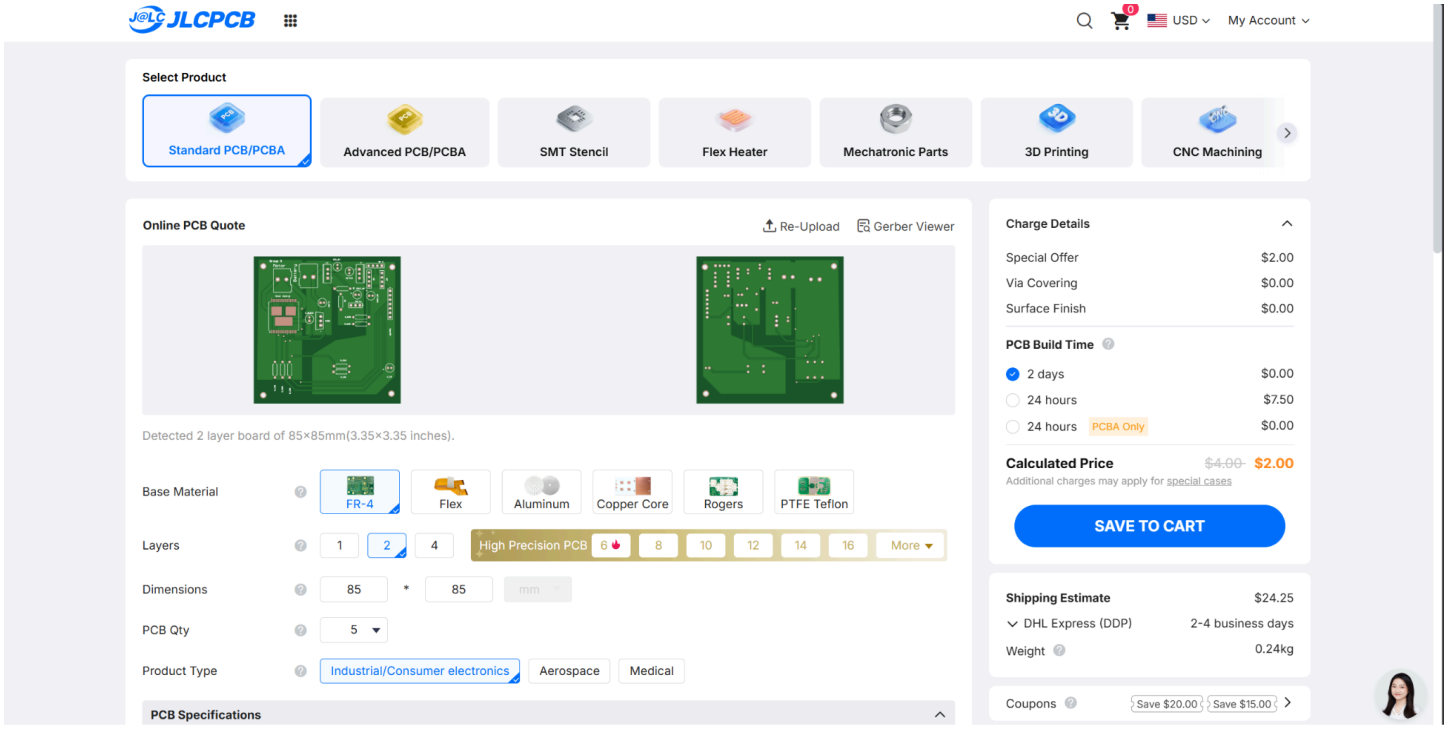
	Min. Distance	Bay Area Circuits Standard Capability	Bay Area Circuits Advanced Capability
copper to board edge:	62.78 mil (1.595 mm)	10 mil	0 mil
copper to plated hole:	16.5 mil (0.419 mm)	10.5 mil	5 mil
copper to non-plated hole:	-	8 mil	4 mil
copper to copper:	10.0 mil (0.254 mm)	6 mil	2 mil
copper ring:	6.5 mil (0.17 mm)	6 mil	1 mil
track width:	16.0 mil (0.406 mm)	6 mil	2 mil

Green = Standard Capability, Orange = Advanced Capability, Red = Not Within Capability

JLC DFM Check:



PCB Price:



Final Report

EEC 195 Team 9 Final Report

Nicolas Wong-Shea, Temirlan Ryspaev, Xiang Mao, Justin Spenker

Table of Contents

I. Project Objective

II. Hardware Documentation

A. Detailed System Block Diagram

B. OpenMV Interface and Circuit Connections

C. Motor Control PCB Schematic and Layout

D. Mechanical Design and Mounting Hardware

III. Software Documentation

A. High-Level Code Description

B. Main Software Modules

C. ROI and Lane Detection Strategy

D. PD Control and Speed Strategy

E. Pseudo-Code of the Final Software Loop

F. Debugging and Validation Features

IV. Pictures of the Car

V. Design and Performance Summary

VI. References and Acknowledgements of Sources

I. Project Objective

The objective of this project was to design, build, and test an autonomous lane-following vehicle that combines custom hardware, embedded vision processing, and real-time closed-loop motor control. The vehicle uses an OpenMV camera to observe the track, detect lane markings, estimate the vehicle position relative to the lane center, and generate steering and speed commands without direct human control. The project therefore demonstrates a practical integration of electrical design, computer vision, software control, and mechanical mounting on a small autonomous vehicle platform.

A second objective was to create a reliable hardware platform that could support high-current motor operation while still providing stable power and clean control signals for sensitive logic devices. To achieve this, the team designed a custom two-layer motor control PCB, routed the battery and motor current paths, included regulated power rails for the control electronics, and integrated connectors for the OpenMV camera, steering servo, DC motor driver, and supporting components. The final design goal was not only to make the car move, but to make the entire hardware and software system work together reliably at speed.

II. Hardware Documentation

A. Detailed System Block Diagram

The autonomous vehicle can be divided into three major subsystems: the power and motor control hardware, the OpenMV vision and software subsystem, and the mechanical vehicle platform. The OpenMV camera captures track images and computes control outputs. The custom PCB distributes power, routes control signals, and drives the motor hardware. The vehicle chassis, steering servo, DC motor, and camera mount translate those electrical commands into physical motion.

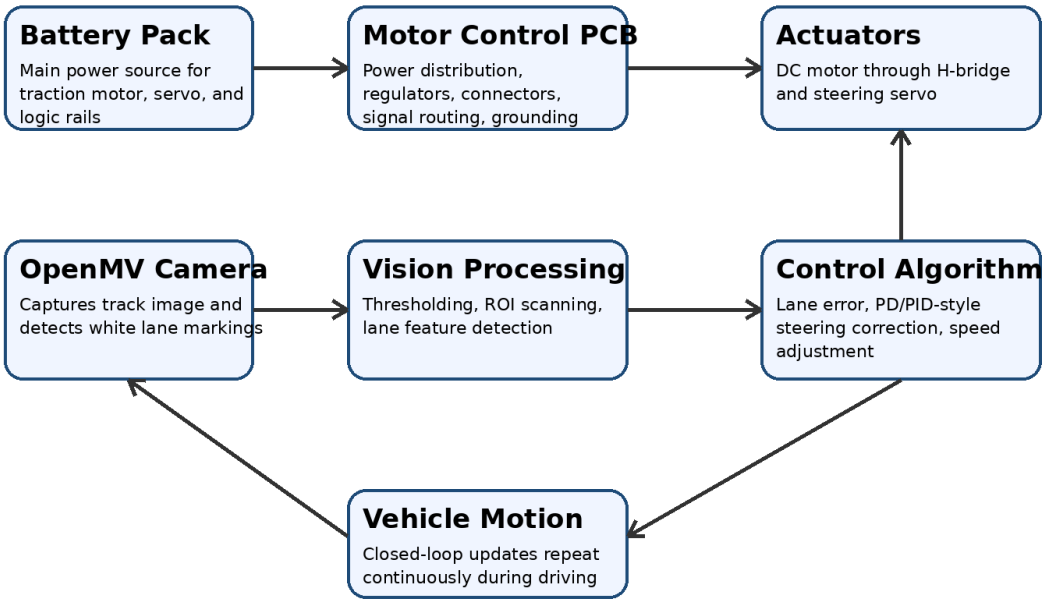


Figure 1. System block diagram showing the relationship between the battery, custom PCB, OpenMV camera, vision processing, control algorithm, motor driver, steering servo, and vehicle motion.

Subsystem	Main Components	Purpose
Power subsystem	Battery pack, voltage regulators, PCB power traces, ground routing	Provides stable voltage rails and high-current paths for the motor, servo, OpenMV, and logic components.
Motor control subsystem	VNH5019-style H-bridge motor driver, PWM/direction signals, DC motor connections	Converts low-power control signals into high-current drive for the propulsion motor.
Steering subsystem	Steering servo and PWM signal connection	Controls the front wheel angle or steering linkage based on the lane-following correction.
Vision subsystem	OpenMV camera module and related signal/power connections	Captures real-time images of the track and computes the lane-following error.
Mechanical subsystem	Vehicle chassis, PCB mounting, OpenMV camera mount, wiring support	Physically supports the electronics and keeps the camera view stable during motion.

B. OpenMV Interface and Circuit Connections

The OpenMV camera acts as the main perception and control device for the vehicle. It receives regulated power from the PCB and sends control signals to the motor control and steering hardware. Since the lane-following behavior depends on continuous image capture and fast control-loop updates, the OpenMV connections must be mechanically secure and electrically stable.

The main OpenMV-related circuit connections are summarized below. The exact pin names should be updated to match the final schematic export before submission.

OpenMV Connection	Connected Hardware	Function
Regulated power input	5 V or appropriate regulated supply from the PCB	Powers the OpenMV camera and prevents unstable operation caused by battery voltage variation.
Ground	PCB ground network	Provides a common voltage reference for OpenMV, motor driver logic, servo signal, and regulators.
Servo PWM output	Steering servo signal pin	Sends steering commands generated by the lane-following control algorithm.
Motor PWM / direction outputs	Motor driver control inputs	Commands the motor driver to control propulsion speed and direction.
Optional debug or serial pins	Computer, display, or debug interface	Supports testing, parameter tuning, and troubleshooting during development.

One design decision was to keep the control wiring organized and separated from high-current motor traces as much as possible. DC motors and servos can introduce electrical noise, especially during acceleration, braking, or sharp steering changes. For this reason, the PCB layout and wiring arrangement were designed to reduce noise coupling into the OpenMV and logic signals.

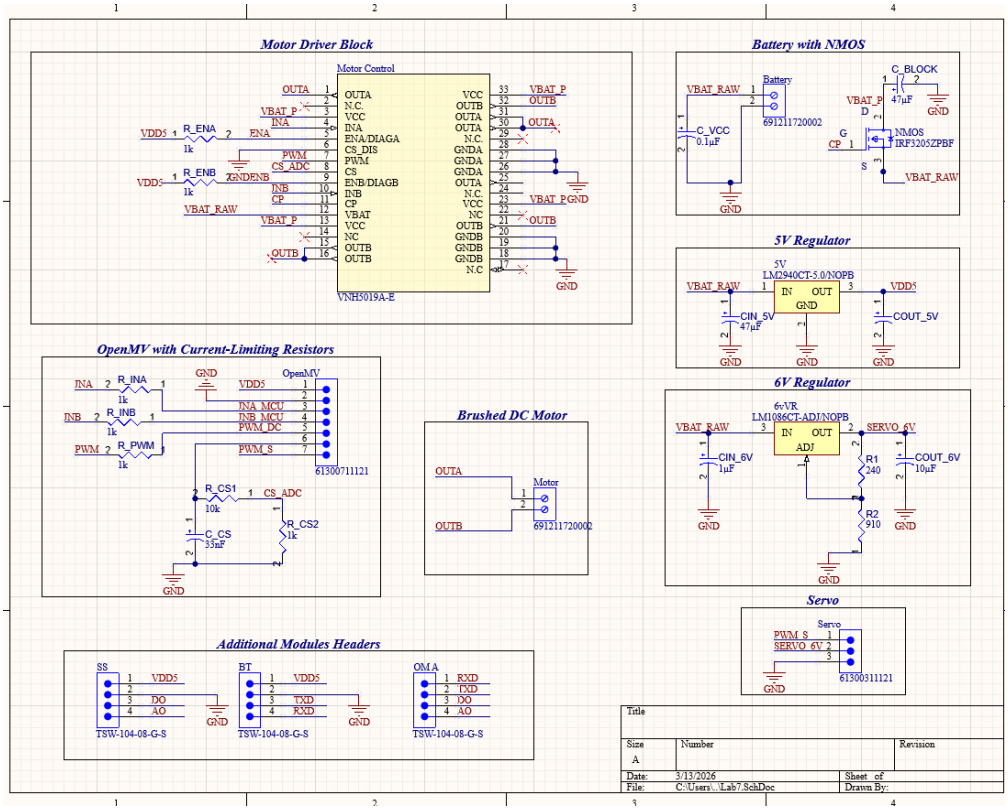


Figure 2. Placeholder for the final OpenMV interface schematic and related circuit connections.

C. Motor Control PCB Schematic and Layout

The custom motor control PCB was designed as a compact two-layer board that integrates the main power and control connections needed by the autonomous vehicle. The board includes motor driver circuitry, regulator connections, servo connections, OpenMV connections, and external headers. The purpose of the PCB was to reduce wiring complexity, improve reliability, and provide a cleaner hardware platform compared with loose breadboard-style wiring.

The propulsion motor is driven through a surface-mount H-bridge motor driver. This driver allows the low-power control side to command a much larger motor current, which is necessary for the vehicle to accelerate and maintain speed around the track. The PCB layout includes wider traces for the high-current path from the battery to the motor driver and motor output. Control and logic traces are routed separately where possible.

The PCB design also includes regulated voltage rails for the logic and actuator subsystems. In the vehicle, different components require different supply characteristics. The OpenMV and logic circuitry require stable regulated power, while the motor and servo can draw larger and more variable currents. Separating these functions helps prevent resets, unstable camera behavior, or noisy control signals during aggressive driving.

PCB Design Feature	Reason for Including It	Engineering Benefit
Two-layer board layout	Allows compact routing while keeping the board manufacturable for a class project.	Reduces loose wiring and improves system reliability.
Localized/star-style grounding approach	Keeps high-current return paths from strongly disturbing sensitive control references.	Improves signal stability for the OpenMV and servo control lines.
High-current motor traces	Motor current is much larger than logic current.	Reduces voltage drop and lowers the risk of overheated or undersized traces.
Dedicated regulator sections	OpenMV, logic, and actuators require stable power.	Reduces brownouts and electrical noise problems.
Connector-based integration	The vehicle needs to be assembled, tested, and repaired quickly.	Improves debugging, replacement, and final demonstration readiness.

During PCB development, one major challenge was balancing compactness with manufacturability. A compact board saves space on the vehicle, but dense routing makes soldering, inspection, and debugging more difficult. The team reduced trace complexity where possible, checked clearances around high-current pads, and considered how the board would be mounted and accessed after assembly.

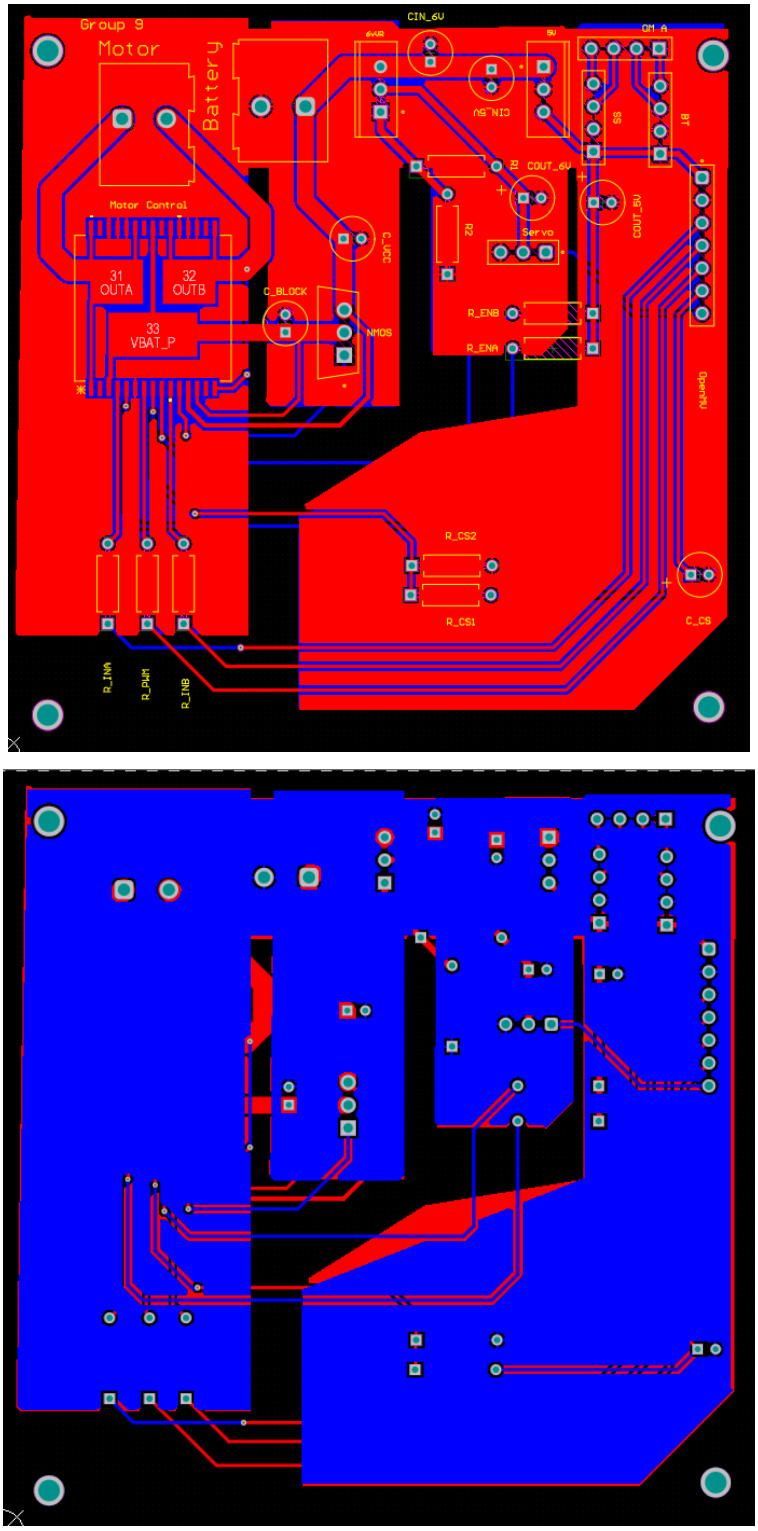


Figure 3. Placeholder for the final motor control PCB layout export. Replace with top/bottom copper and silkscreen images if available.

D. Mechanical Design and Mounting Hardware

Mechanical integration was important because the camera position directly affects the quality of the lane detection. If the camera is mounted too low or aimed too close to the vehicle, the software reacts late to upcoming turns. If the camera is mounted too high or aimed too far away, the lane markings may become smaller and more difficult to detect. The team therefore treated the OpenMV mount as part of the control system rather than as a simple mechanical accessory. The camera mounting design focused on three requirements: stable support, adjustable viewing angle, and enough clearance above the chassis. A stable mount prevents vibration from blurring the image or changing the ROI location during driving. An adjustable angle allows the team to tune how far ahead the camera can see. Proper clearance ensures that the camera view is not blocked by the car body, wires, battery, or PCB.

Mechanical Requirement	Design Choice	Effect on Performance
Stable camera view	Rigid mounting structure and secure fasteners	Reduces image jitter and keeps ROI regions aligned with the track.
Adjustable camera angle	Mounting geometry that allows the OpenMV to be tilted during testing	Allows tuning between near-field stability and far-field curve anticipation.
PCB protection	Board mounted away from moving parts and excessive flex	Improves electrical reliability and reduces the chance of wire or component damage.
Serviceability	Accessible connectors and mounting hardware	Makes it easier to change wiring, debug connections, and repair after crashes.

Future mechanical improvements could include a more rigid camera mast, vibration damping, clearer strain relief for wires, and a more repeatable method for adjusting the OpenMV angle. These improvements would make tuning easier and would reduce run-to-run variation during testing.

III. Software Documentation

A. High-Level Code Description

The final software runs directly on the OpenMV camera as a real-time closed-loop lane follower. At startup, the program initializes the camera, motor direction pins, steering PWM output, drive PWM output, grayscale image format, image resolution, white-line threshold, and steering calibration values. During operation, the software repeatedly captures an image, detects the lane boundaries, estimates the driving error, computes a steering correction, selects a motor speed, and sends PWM commands to the vehicle actuators.

The vision system uses grayscale images and a brightness threshold to emphasize the white lane markings on the track. Instead of searching the entire frame equally, the image is divided into a left region of interest and a right region of interest. The software runs line regression inside each side region to estimate the lane boundaries. This approach focuses computation on the useful part of the image and allows the program to treat the left and right lane markings separately.

The lane estimator supports several operating modes. When both lane boundaries are visible, the software directly estimates the lane center by averaging the left and right line positions. When only one lane boundary is visible, the software estimates the missing boundary using a filtered lane-width value. This single-line recovery behavior helps the car continue through turns when one side of the lane temporarily leaves the camera view. When no reliable lane line is detected, the software slows the car and briefly fades the previous error estimate to avoid a sudden uncontrolled response.

The steering control uses a proportional-derivative controller. The proportional term reacts to the current lane error, while the derivative term reacts to how quickly that error is changing. The derivative calculation is filtered so that small frame-to-frame image noise does not cause

unstable steering. The final software does not use any additional accumulated-error control term, which makes the steering response faster and reduces overshoot after leaving curves.

The speed logic is connected to the steering demand. When the required steering correction is small, the car uses a faster motor PWM value for straight sections. As the required correction increases, the software reduces the motor PWM toward a slower value. If a curve is detected, the software forces a slower turning speed. This strategy allows the vehicle to maintain speed on straight sections while improving stability during sharp turns.

B. Main Software Modules

Software Module	Main Responsibility	Important Output
Camera initialization	Configure image format, resolution, exposure behavior, and camera parameters.	Stable input frames for processing.
Thresholding	Convert the grayscale image into a form where white lane markings can be separated from the track.	Filtered image emphasizing lane markings.
Left and right ROI scanning	Search selected side regions instead of the entire frame.	Detected left and right lane line candidates.
Line regression	Estimate the position and direction of the left and right lane markings.	Left and right lane vectors.
Error calculation	Compare the estimated lane center and heading direction with the image center.	Signed steering error.
Control calculation	Apply proportional and derivative terms to generate the steering correction.	Steering correction value.
Speed adjustment	Reduce speed when the steering correction becomes large or a curve is detected.	Motor PWM or speed command.
Actuator output	Send PWM commands to the steering servo and motor driver.	Vehicle motion update.

C. ROI and Lane Detection Strategy

ROI / Detection Type	What It Detects	How It Helps the Car
Left ROI	White lane marking on the left side of the image.	Helps estimate the left lane boundary.
Right ROI	White lane marking on the right side of the image.	Helps estimate the right lane boundary.
Near reference point	Lane position close to the vehicle.	Improves immediate centering and short-term stability.
Far reference point	Lane trend farther ahead.	Helps the car anticipate curves before entering them.
Both-lane detection	Both left and right lane markings are detected.	Provides the most reliable lane center estimate.
Single-lane detection	Only one lane marking is detected.	Allows the car to estimate the lane center using the stored lane width.
Lost-line handling	No reliable lane marking is detected.	Reduces speed and uses the previous error briefly for safer recovery.

D. PD Control and Speed Strategy

Control Term / Strategy	Role in the Vehicle	Risk if Poorly Tuned
Proportional term (P)	Turns the vehicle in response to the current lane error.	Too low: slow correction. Too high: aggressive oscillation.
Derivative term (D)	Damps rapid error changes and reduces left-right shaking.	Too low: weak damping. Too high: noisy steering response.
Curve detection	Detects sharp turns using lane mode and heading error.	Too insensitive: car enters curves too fast. Too sensitive: car slows too often.
Minimum curve correction	Forces a minimum steering response when a curve is detected.	Too low: car may not turn enough. Too high: car may oversteer.
Dynamic speed reduction	Slows the car for large steering corrections or sharp curves.	Too little: car leaves the lane. Too much: lap time becomes slow.
Smooth steering update	Limits sudden changes in the steering PWM command.	Too slow: delayed turning. Too fast: unstable steering motion.

E. Pseudo-Code of the Final Software Loop

```
initialize OpenMV camera in grayscale QQVGA mode
initialize motor direction pins, steering PWM, drive PWM, thresholds, and side ROIs
set steering to straight position
start motor with a short kickstart

while vehicle is running:
    image = capture_frame()

    left_line, right_line = detect_lane_lines(image, left_ROI, right_ROI)
    cte, heading, center_x, heading_angle, mode = compute_lane_error(left_line,
right_line)

    if mode is LOST:
        cte = previous_cte * recovery_decay
        heading = previous_heading * recovery_decay
        speed = slow_speed
    else:
        save current cte and heading for possible recovery

    if both lane lines are detected:
        combined_error = cte_weight * cte + heading_weight * heading
    else if one lane line is detected:
        estimate the lane center using the filtered lane width
        enforce a minimum correction toward the visible lane boundary
    else:
        combined_error = cte_weight * cte + heading_weight * heading

    correction = PD_controller(combined_error)

    if curve is detected and correction is too small:
        apply minimum curve correction

    speed = dynamic_speed(abs(correction))
    if curve is detected:
        speed = slow_turning_speed

    target_servo_pulse = straight_servo_pulse + correction * steering_range
    target_servo_pulse = clamp(target_servo_pulse, left_limit, right_limit)
    actual_servo_pulse = smooth_steering_update(target_servo_pulse)

    send_servo_PWM(actual_servo_pulse)
    send_motor_PWM(speed)
```

F. Debugging and Validation Features

The code includes a debug mode for validation. In debug mode, the program can test steering limits, run simple motor checks, print controller outputs for representative errors, and display the selected speed for different correction magnitudes. The debug drawing function can also mark the ROIs, detected lines, lane center, steering angle, speed label, cross-track error, and heading error on the OpenMV image. These features helped the team tune the camera angle, threshold, ROI placement, steering gain, and speed thresholds during testing.

IV. Pictures of the Car

This section should contain several clear pictures of the final vehicle. The placeholders below should be replaced with final photos before submission. Recommended photos include: full vehicle side view, top-down electronics view, close-up of the OpenMV mount, close-up of the PCB, and one photo of the car on the track.

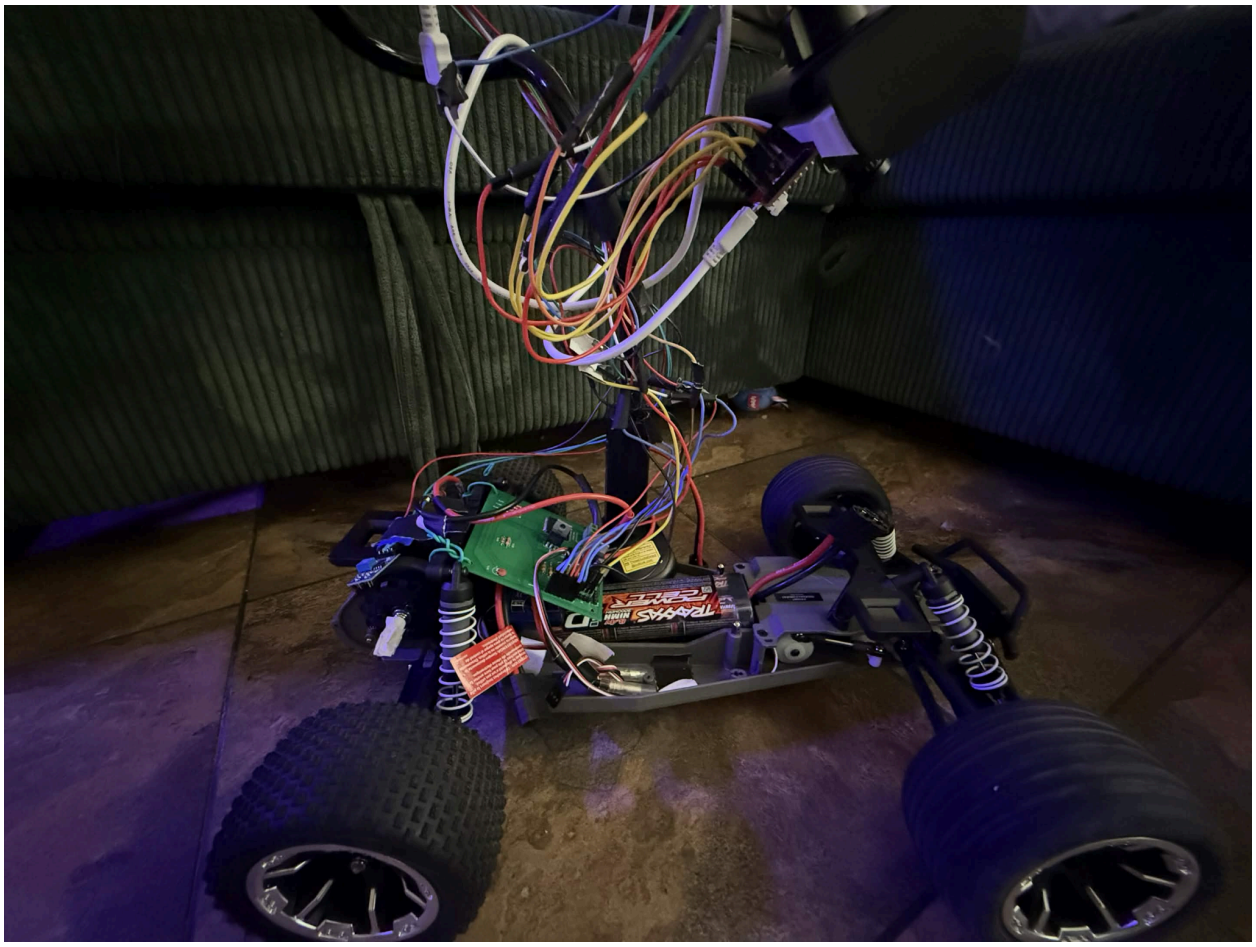


Figure 4. Placeholder for a full front/side vehicle photo.

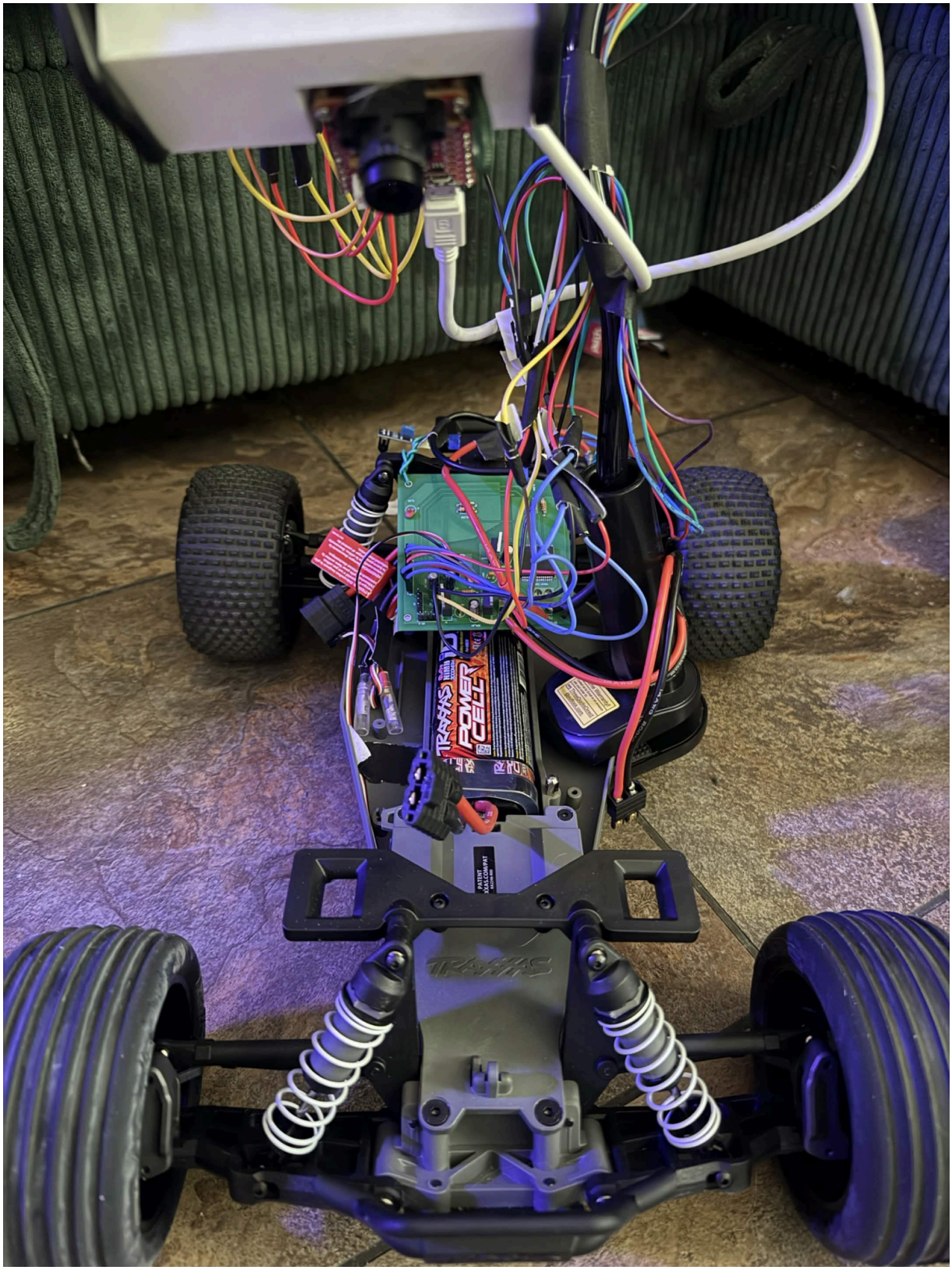


Figure 5. Placeholder for a top-down vehicle integration photo.

V. Design and Performance Summary

The final vehicle design integrated a custom motor control PCB, an OpenMV-based vision processing system, closed-loop steering control, and mechanical mounting components. The custom PCB reduced messy wiring and provided a more organized platform for connecting the battery, motor driver, voltage regulators, steering servo, and camera. The OpenMV software detected lane markings in real time and converted image-based lane information into steering and speed control commands.

The greatest challenge in the design was balancing speed and stability. At low speeds, the vehicle could tolerate slower image updates, small detection errors, and conservative steering actions. However, at higher speeds, even small errors became significant because the vehicle traveled a longer distance between camera frames and steering corrections. Therefore, camera angle, region of interest placement, threshold tuning, and controller gain selection were all critical to vehicle performance. Hardware reliability also directly affected software performance. Even if the lane-following algorithm was logically correct, noise from the DC motor and steering servo, unstable power rails, loose connectors, or poor grounding could appear as control problems.

During the final time trial, our vehicle was not able to achieve the desired balance between speed and accuracy. One likely issue was that the OpenMV mounting structure was not stable enough, which caused the camera view to shake while the vehicle was moving. The OpenMV camera angle may also not have provided enough forward-looking distance, so when the speed increased, the vehicle did not detect turns early enough and the steering corrections became less precise. In addition, the final software used several conservative design choices. For example, the vehicle forced a slower speed when a curve was detected, the curve detection threshold was

relatively sensitive, the look-ahead distance was limited, single-lane mode could force a larger correction, and large steering corrections caused the vehicle to reduce speed. These choices improved stability and helped prevent the vehicle from leaving the lane, but they also limited the maximum speed during competition. Future improvements could include a more rigid camera mount, a better camera angle, longer look-ahead vision, less conservative curve detection, and more advanced speed control for straight sections, curves, uphill sections, and downhill sections.

VI. References and Acknowledgements of Sources

The team would like to thank Professor Halstead and Ning Miao for their instruction, feedback, and support throughout the project. Their guidance helped the team debug both hardware and software issues and improve the final vehicle integration.

References and sources used during design and development should be listed below. Replace placeholder entries with the exact links or datasheets used by the team before submission.

1. OpenMV documentation and camera module reference materials. [https://docs.openmv.io/]
2. PCB design tool documentation or manufacturing rules used for trace width, clearance, and layout checks. [https://www.altium.com/]

Video Script

Tem:

Hello, we are Team 9, and our senior design project is an autonomous lane-following vehicle.

Our system combines a custom motor-control PCB, an OpenMV camera, real-time image processing, closed-loop steering control, and mechanical integration on a small vehicle platform.

The camera observes the track, the software estimates the vehicle's position within the lane, and the PCB delivers the resulting commands to the steering servo and propulsion motor.

My main contribution was designing the electrical schematic and PCB layout alongside Nicolas.

The schematic organized the vehicle into several major sections: the battery input, motor driver, voltage regulators, OpenMV interface, steering servo, propulsion motor, and supporting connectors.

One of our most important design challenges was supplying high motor current while maintaining stable power and clean signals for the OpenMV camera and other control electronics. Motors and servos can produce electrical noise and sudden changes in current, which can interfere with sensitive logic circuits or cause unstable behavior.

To address this, we used separate regulated power sections and a localized grounding approach intended to reduce interference between the high-current and low-power portions of the system.

For the two-layer PCB layout, we used wider traces for the battery and motor-current paths. Some of the main current paths were approximately one hundred mils wide and narrowed near the surface-mount motor-driver pads, where additional clearance was needed.

We also tried to keep control signals separated from the high-current motor paths wherever possible.

The board needed to be compact enough to fit on the vehicle, but making it too dense would make manufacturing, soldering, inspection, and debugging more difficult. Balancing those requirements was one of the central engineering problems in the hardware design.

Nicolas:

After finishing the schematic and PCB layout, the design was sent out for manufacturing.

My main contribution was assembling the manufactured board and integrating it into the vehicle. I soldered both the through-hole and surface-mount components, including the motor-driver circuitry, voltage-regulation components, resistors, capacitors, headers, and external connectors.

Surface-mount assembly was the most challenging part of this process, particularly because of the VNH motor-driver chip's small, closely spaced pins and its placement in a relatively dense area of the board. In order to get access to necessary equipment, I had to get certification from the ESDC for soldering and reflow.

The original plan was to use the pick and place machine to dispense solder paste accurately onto each pad and the large heat slug under the chip. However, the machine was not functioning at the time, so I was forced to manually apply solder paste with a syringe before reflowing the board in the reflow oven.

Because the syringe was much less precise than the pick and place dispenser, the reflow process left several pins with excess solder and bridging. To fix this, I manually touched up every pin by using solder wick to get rid of bridges, then flux and fresh solder to solidify connections. Afterwards, I visually inspected the chip and performed continuity tests across all of its pins, to confirm functionality. The through-hole components were significantly easier to solder, but the sheer number of components made it time consuming.

After assembly, we connected the battery, motor, steering servo, OpenMV camera, and supporting hardware. Connector placement and accessibility mattered because we needed to repeatedly disconnect components while testing and debugging the system.

This stage demonstrated that a schematic and PCB layout are only part of the design process. A successful board must also be manufacturable, solderable, testable, and mechanically compatible with the complete system.

The assembled PCB gave us a cleaner and more reliable platform than that of a simple breadboard and allowed the hardware and software portions of the project to operate as one integrated vehicle.

Justin:

I was focused mainly on developing the lane-following software that runs on the OpenMV camera.

The program operates as a continuous closed-loop system. During each loop, the camera captures an image, detects the lane boundaries, estimates the vehicle's position and heading, calculates a steering correction, selects an appropriate motor speed, and sends PWM commands to the steering servo and motor driver.

The camera uses grayscale images and a brightness threshold to identify the white lane markings against the darker track.

Instead of processing every part of the image equally, the software searches separate left and right regions of interest. Line regression is then used to estimate the location and direction of the lane boundaries.

When both lane markings are visible, the software calculates the lane center using both lines. When only one marking is visible, it estimates the missing side using a filtered lane-width value. This allows the vehicle to continue following the track when one line temporarily moves outside the camera's field of view.

The steering system uses proportional and derivative control. The proportional term responds to the current lane error, while the derivative term reduces rapid oscillation by responding to how quickly that error is changing.

The software also changes the motor speed based on steering demand. The vehicle travels faster when the correction is small and slows down when it detects a curve or requires a larger steering correction. This improves stability while still allowing greater speed on straight sections.

Xiang:

I assisted with the software and system testing, particularly with understanding how the vehicle behaved when lane detection became unreliable.

The software includes several operating conditions. The most reliable condition occurs when both lane boundaries are visible. When only one boundary is detected, the program estimates the lane center based on the expected lane width.

If neither lane line is detected, the vehicle enters a recovery condition. It reduces its speed and briefly uses a decaying version of its previous steering error. This prevents the steering command from changing suddenly when the camera temporarily loses the track.

The program also included debugging features that displayed the regions of interest, detected lane lines, estimated lane center, cross-track error, heading error, steering correction, and selected motor speed.

These outputs helped the team distinguish between several possible causes of poor performance. A steering problem could come from the controller gains, but it could also result from camera vibration, poor camera angle, changing lighting, electrical noise, or inconsistent lane detection.

During the final time trial, the vehicle did not reach the balance between speed and accuracy that we wanted. At higher speeds, the camera did not always identify turns early enough, and movement in the camera mount made the image less stable.

Based on our testing, the most valuable future improvements would be a more rigid camera mount, a longer visual look-ahead distance, and less conservative speed control for straight sections and gradual curves.